

Database Management System

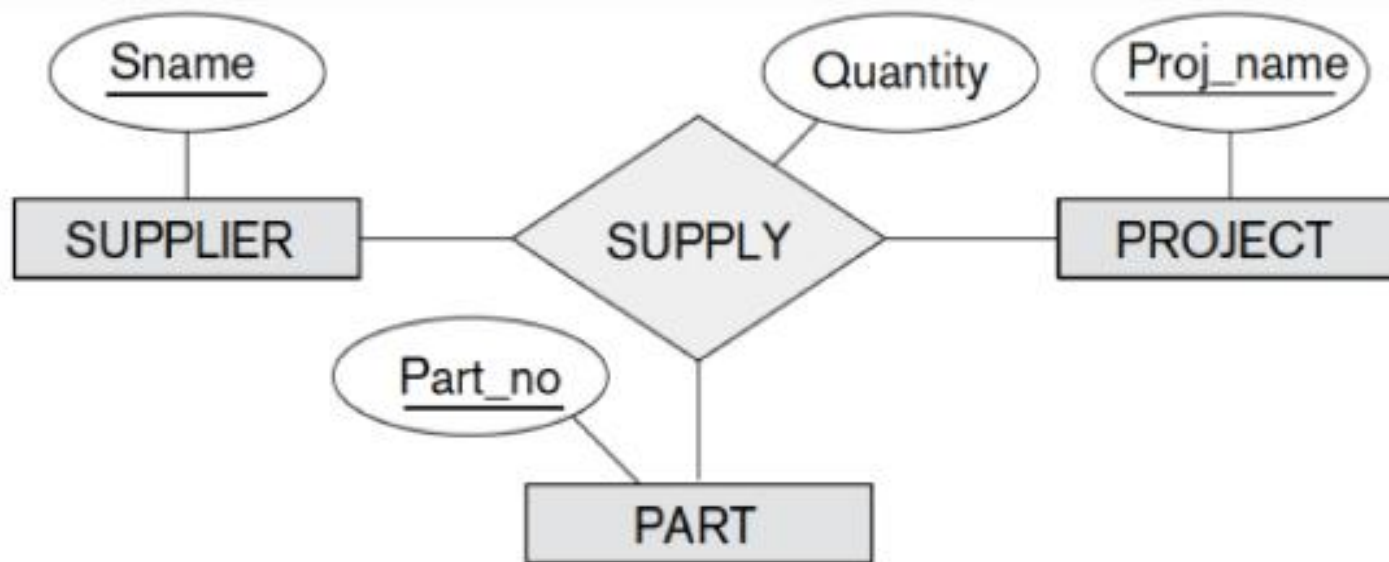
Subject Code : CSC403

Credits : 3

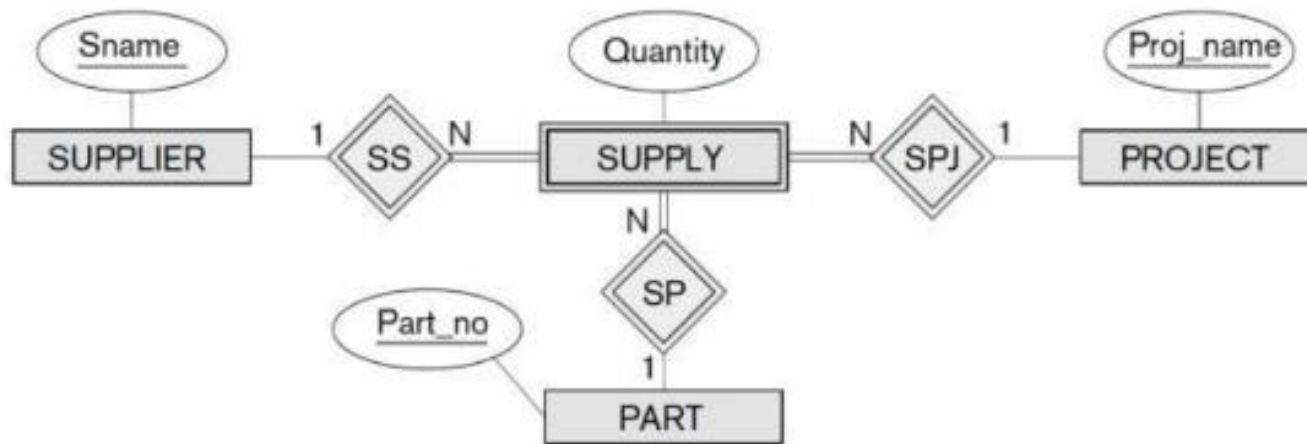
--Shilpa Ingoley

Convert ternary relationship to binary

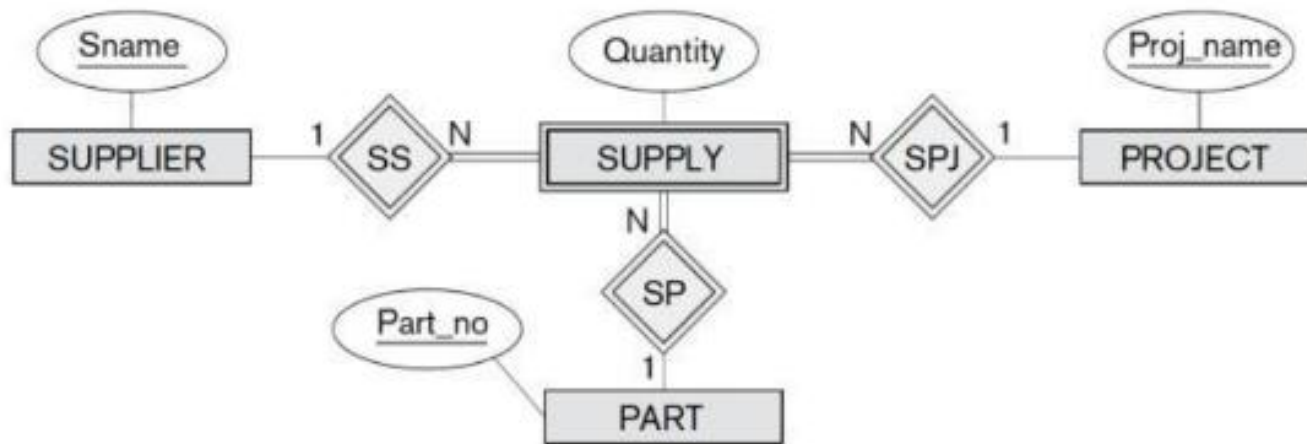
- How to convert ternary relationship to binary. One way is using weak entity relationship as follows (each relationship is M:N cardinality):



- Convert the upper relationship with weak relationship

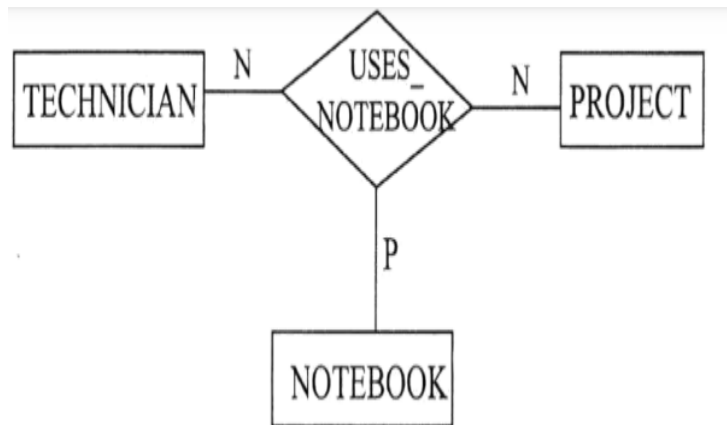


- Convert the upper relationship with weak relationship

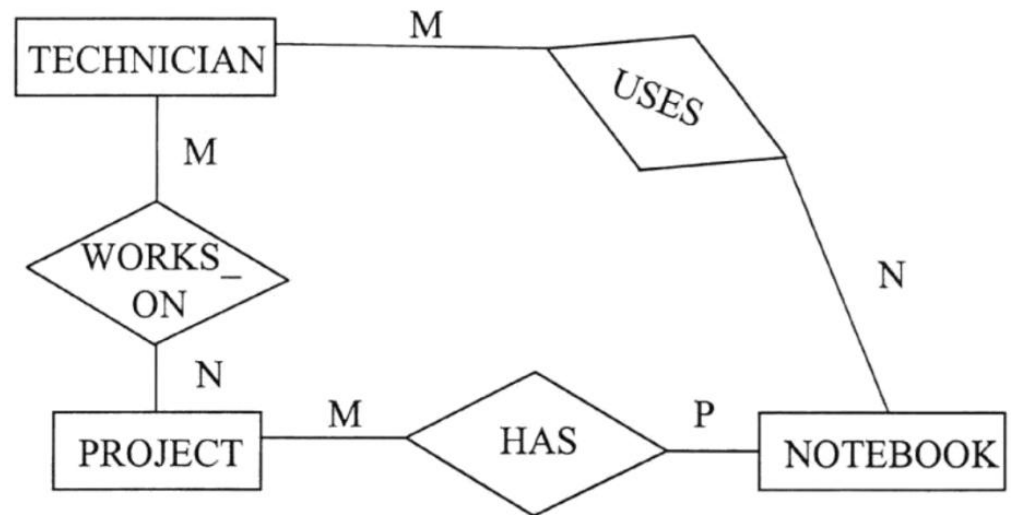


Example 2:

- "if each technician can be working on several projects and uses the same notebooks on each project, then we can decompose 3-ary relationship into binary relationships" as follows:



=> can decompose 3-ary relationship into binary relationships



Module 2: Entity–Relationship Data Model

- The Entity-Relationship (ER) Model: Entity types: weak and strong entity sets, Entity sets, Types of attributes, Keys
- Relationship constraints: Cardinality and Participation
- Extended Entity-Relationship (EER) Model: Generalization, Specialization and Aggregation

Extended Entity Relationship (EER) Diagram

Extended/Enhanced Entity Relationship Model

- Basic ER Model concept were not able to represent requirement of newer and complex applications
- It support additional semantic concepts
- EER model support all the concept of the original ER models together with additional concepts:
 - Super class and subclass Entity -set types
 - Specialization
 - Generalization
 - Attributes Inheritance
 - Categorization
 - Aggregation

Contd...

□ EER = ER Model + Specialization

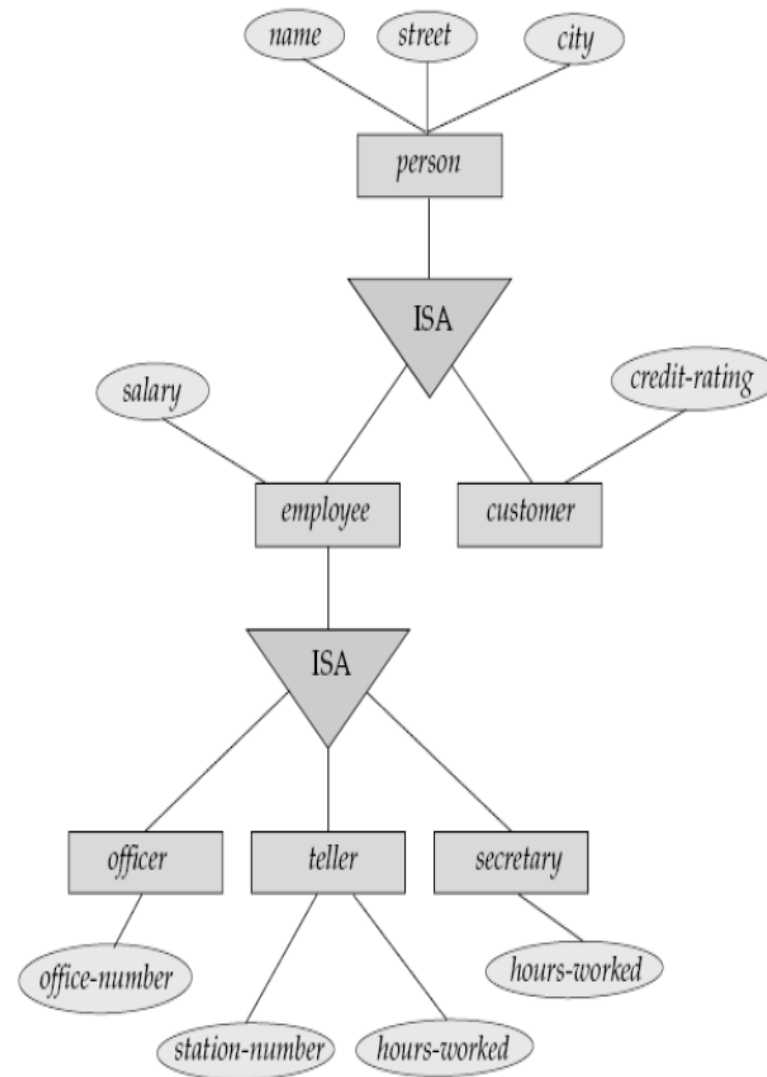
Generalization

Attributes Inheritance

Categorization

Aggregation

Contd...



Attribute Inheritance

- **Attribute Inheritance**

- A crucial property of the higher- and lower-level entities created by specialization and generalization is **attribute inheritance**.
- The attributes of the higher-level entity sets are said to be **inherited** by the lower-level entity sets.
- Eg: *customer* and *employee* inherit the attributes of *person*. Thus, *customer* is described by its *name*, *street*, and *city* attributes, and additionally a *customer-id* attribute

employee is described by its *name*, *street*, and *city* attributes, and additionally *employee-id* and *salary* attributes.

- A lower-level entity set (or subclass) also inherits participation in the relationship sets in which its higher-level entity (or superclass) participates.
- The *officer*, *teller*, and *secretary* entity sets can participate in the *works-for* relationship set, since the superclass *employee* participates in the *works-for* relationship

Specialization

- An entity set may be specialized by more than one distinguishing feature.

Eg: specialization could be based on whether the person is

- *temporary-employee* and
- *permanent-employee*.

- When more than one specialization is formed on an entity set, a particular entity may belong to multiple specializations.

Eg: a given employee may be a temporary employee who is a secretary.

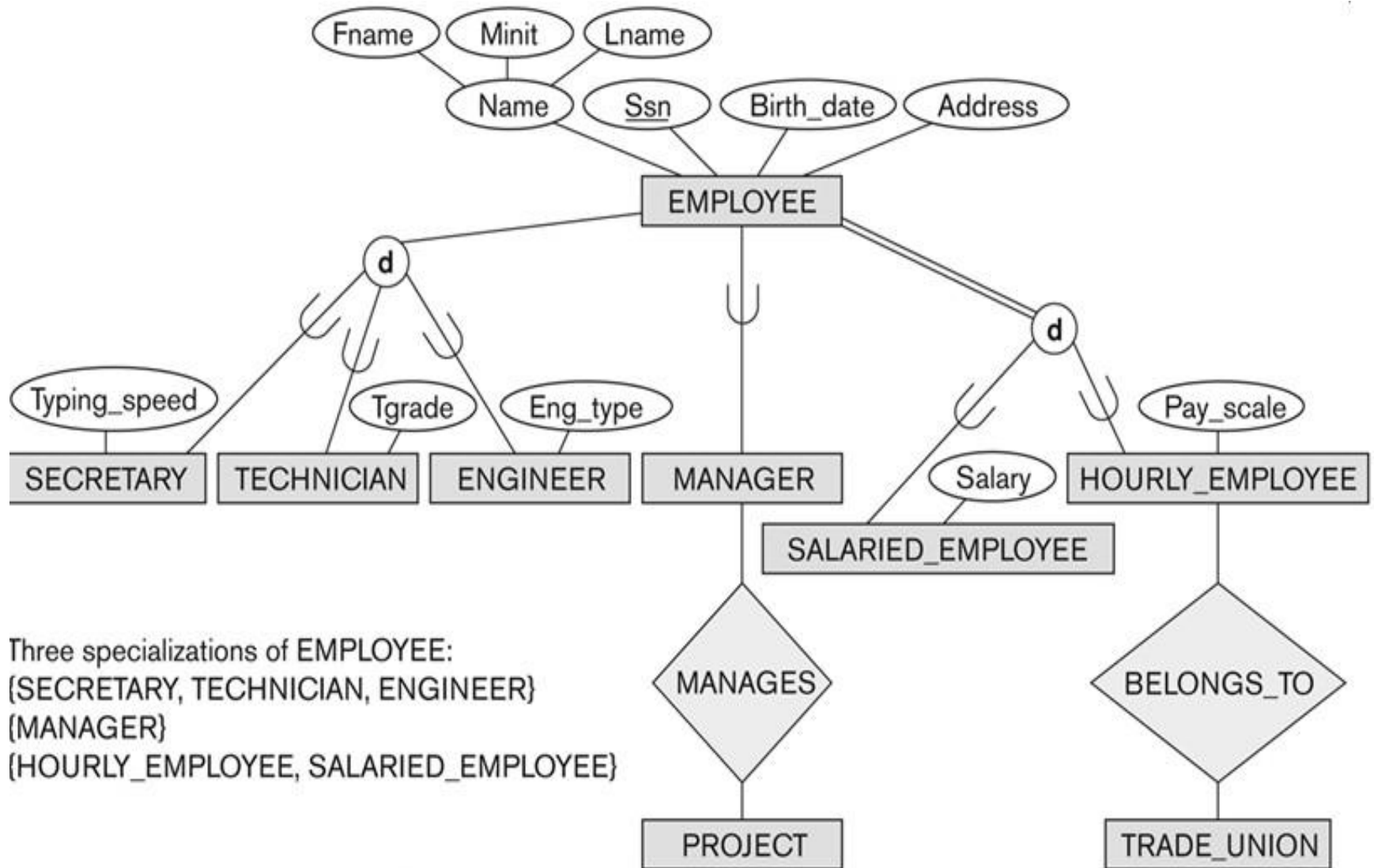
- Specialization is depicted by a *triangle* component labelled **ISA**
- The label ISA stands for “is a” and represents, for example, that a customer “is a” person.
- The ISA relationship may also be referred to as a **superclass-subclass** relationship.
- Higher- and lower-level entity sets are depicted as regular entity a set—that is, as *rectangles* containing the name of the entity set.

Specialization

...

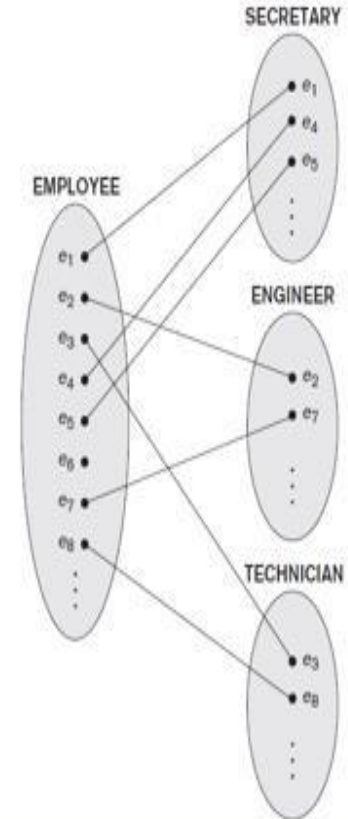
- **Example:** Another specialization of EMPLOYEE based on *method of pay* is {SALARIED_EMPLOYEE, HOURLY_EMPLOYEE}.
 - Superclass/subclass relationships and specialization can be diagrammatically represented in EER diagrams
 - Attributes of a subclass are called ***specific or local attributes***.
 - For example, the attribute TypingSpeed of SECRETARY
 - The subclass can also participate in specific relationship types.
 - For example, a relationship BELONGS_TO of HOURLY_EMPLOYEE

Specialization...



Specialization...

- **Specialization**
 - Process of defining a set of subclasses of an entity type
 - Defined based on some distinguishing characteristic of the entities in the superclass
- Subclass can define:
 - **Specific attributes**
 - **Specific relationship types**

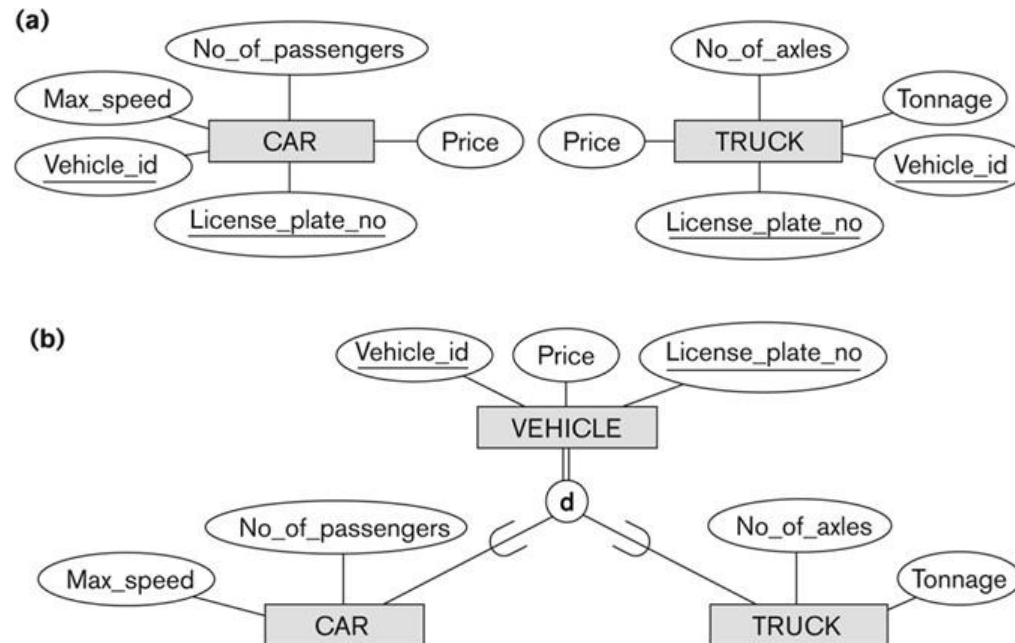


Instances of specialization

Generalization

- The refinement from an initial entity set into successive levels of entity sub-groupings represents a **top-down** design process
- The design process may also proceed in a **bottom-up** manner, in which multiple entity sets are synthesized into a higher-level entity set on the basis of common features.
- The database designer may have first identified a **customer entity set** with the attributes *name*, *street*, *city*, and *customer-id*, and an **employee entity set** with the attributes *name*, *street*, *city*, *employee-id*, and *salary*.
- **Generalization** is a containment relationship that exists between a *higher-level* entity set and one or more *lower-level* entity sets.
- Higher- and lower-level entity sets also may be designated by the terms **superclass** and **subclass**, respectively.

Generalization...



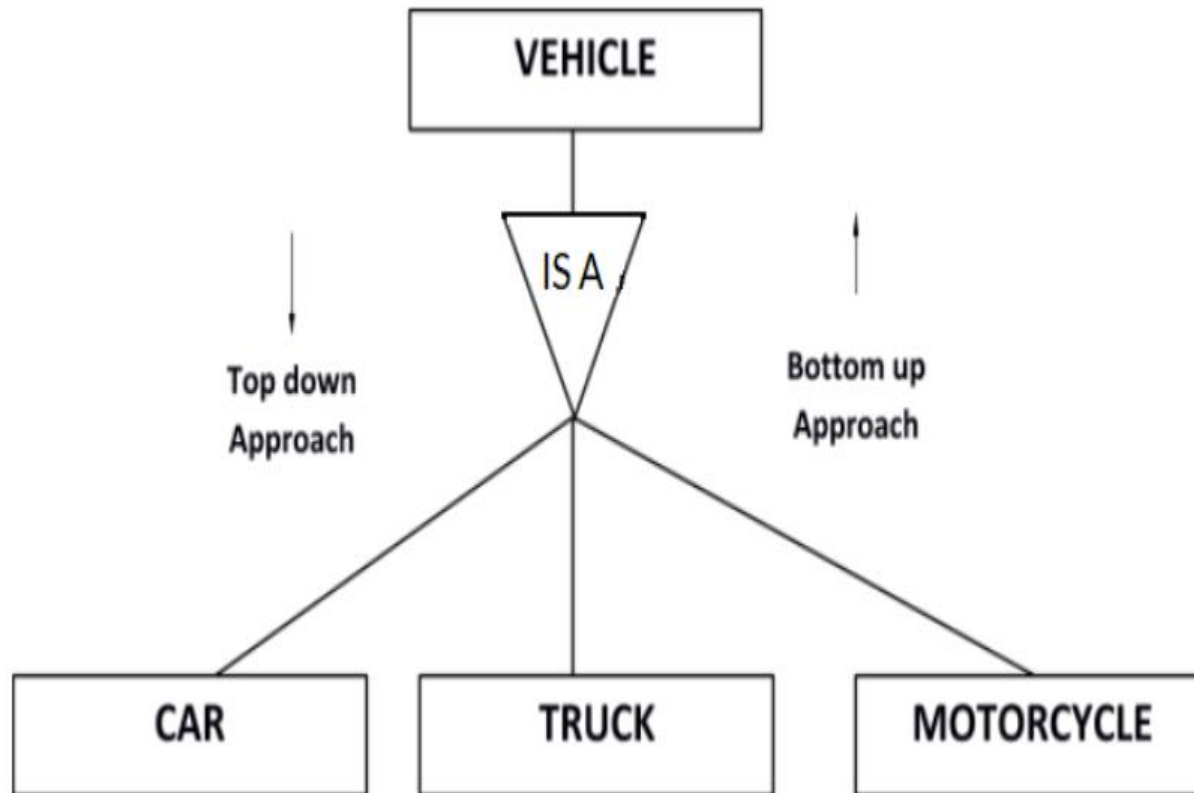
Generalization (a) Two entity types, CAR and TRUCK

(b) Generalizing CAR and TRUCK into the superclass VEHICLE

Specialization and Generalization

- **Generalization** is a process of generalizing an entity which contains generalized attributes or properties of generalized entities.
- It is a **Bottom up** process i.e. consider we have 3 sub entities Car, Truck and Motorcycle. Now these three entities can be generalized into one super class named as Vehicle.
- **Specialization** is a process of identifying subsets of an entity that share some different characteristic. It is a **top down** approach in which one entity is broken down into low level entity.
- In above example Vehicle entity can be a Car, Truck or Motorcycle.

Specialization and Generalization



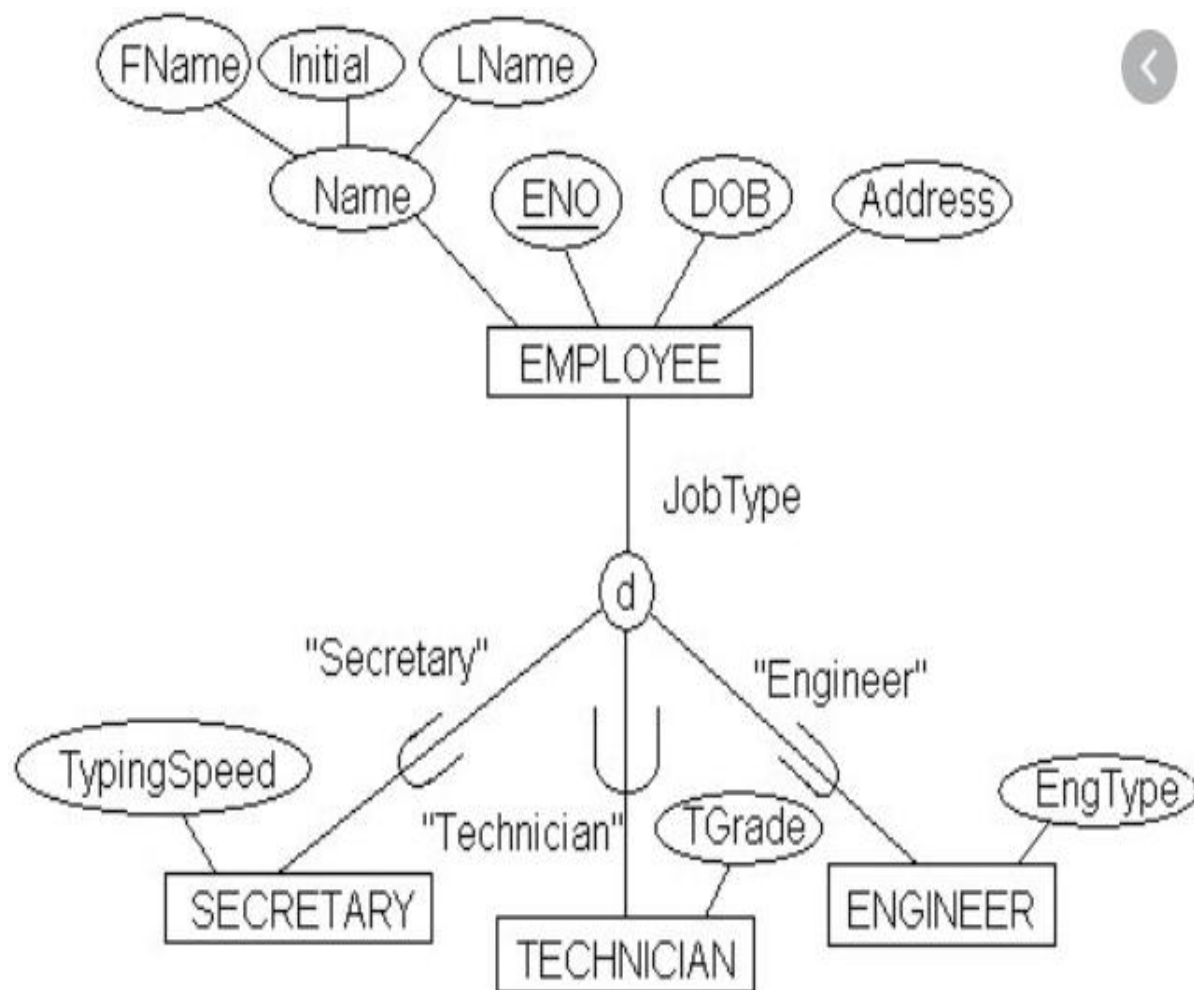
Constraints on Generalization

- **Condition-defined.** In condition-defined lower-level entity sets, membership is evaluated on the basis of whether or not an entity satisfies an explicit condition or predicate. Since all the lower-level entities are evaluated on the basis of the same attribute, this type of generalization is said to be **attribute-defined**.

For example, assume that the higher-level entity set *account* has the attribute *account-type*. All *account* entities are evaluated on the defining *account-type* attribute. Entities that satisfy the condition *account-type* = “savings account” : are allowed to belong to the lower-level entity set *saving account*.

All entities that satisfy the condition

account-type = “checking account” : are included in *checking account*.

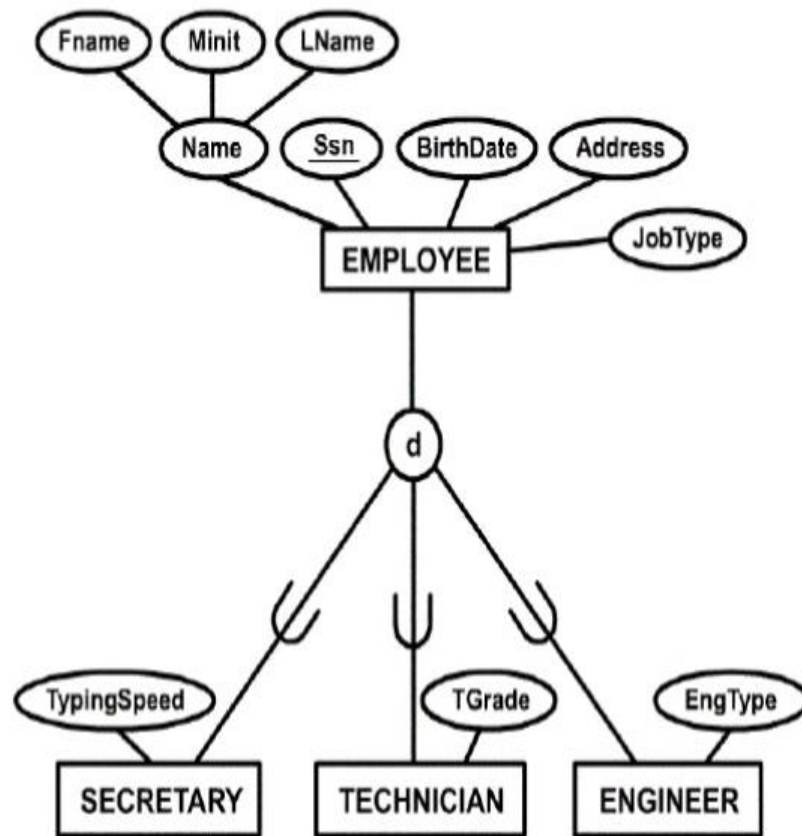


Contd...

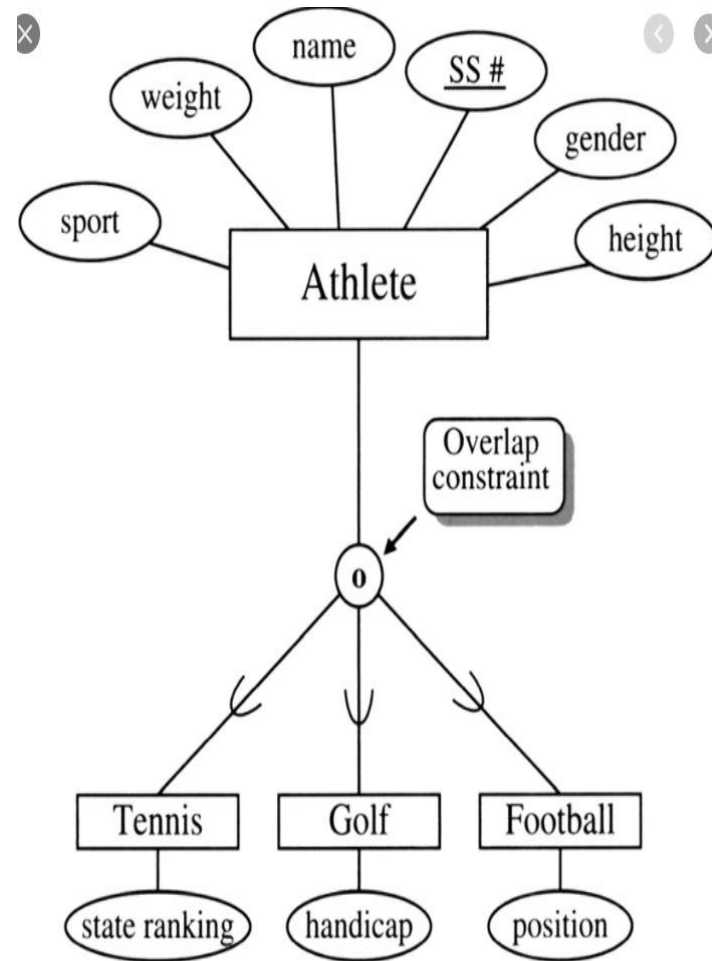
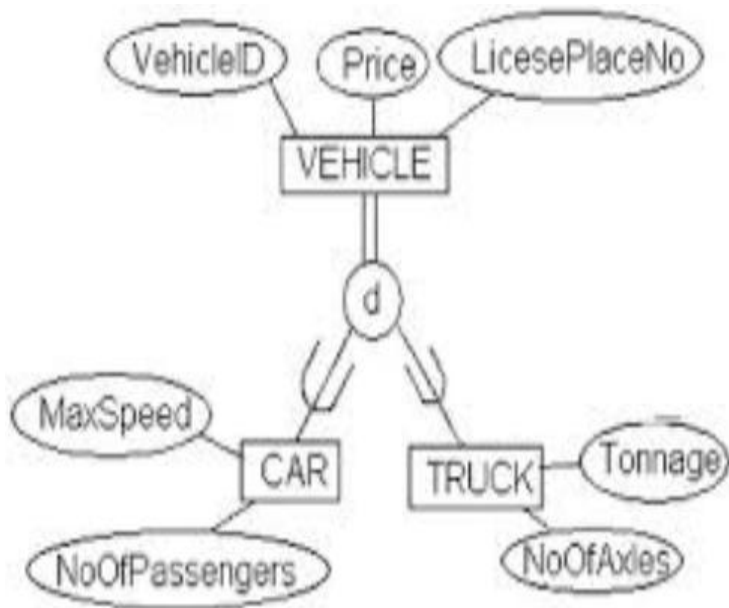
- **User-defined.** User-defined lower-level entity sets are not constrained by a membership condition; rather, the database user assigns entities to a given entity set.
- For instance, let us assume that, after 3 months of employment, bank employees are assigned to one of four work teams.

We therefore represent the teams as four lower-level entity sets of the higher-level *employee* entity set. A given employee is not assigned to a specific team entity automatically on the basis of an explicit defining condition.

Instead, the **user in charge of this decision makes the team assignment** on an individual basis.

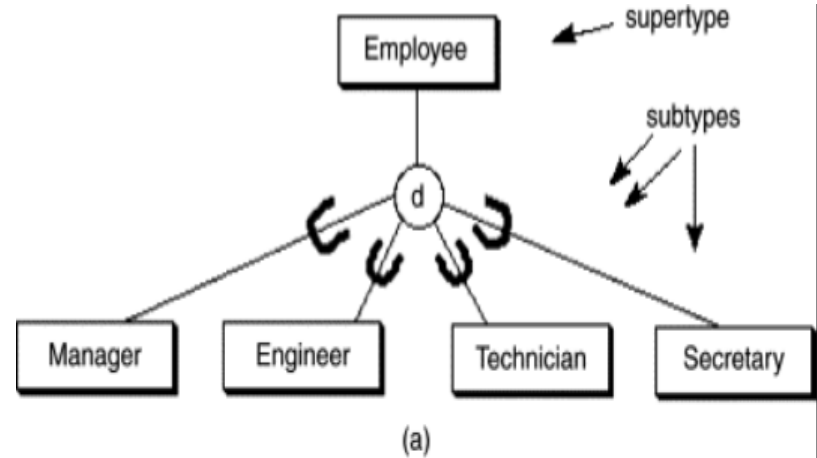


Contd...

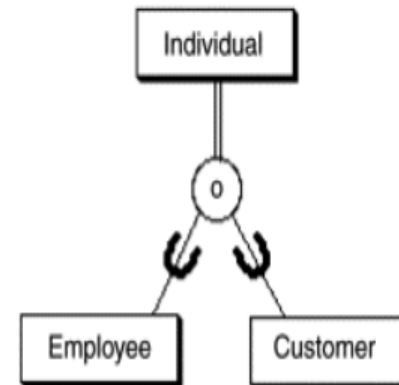


Contd...

- Fig(a): Disjoint



- Fig(b): Overlapping



Contd...

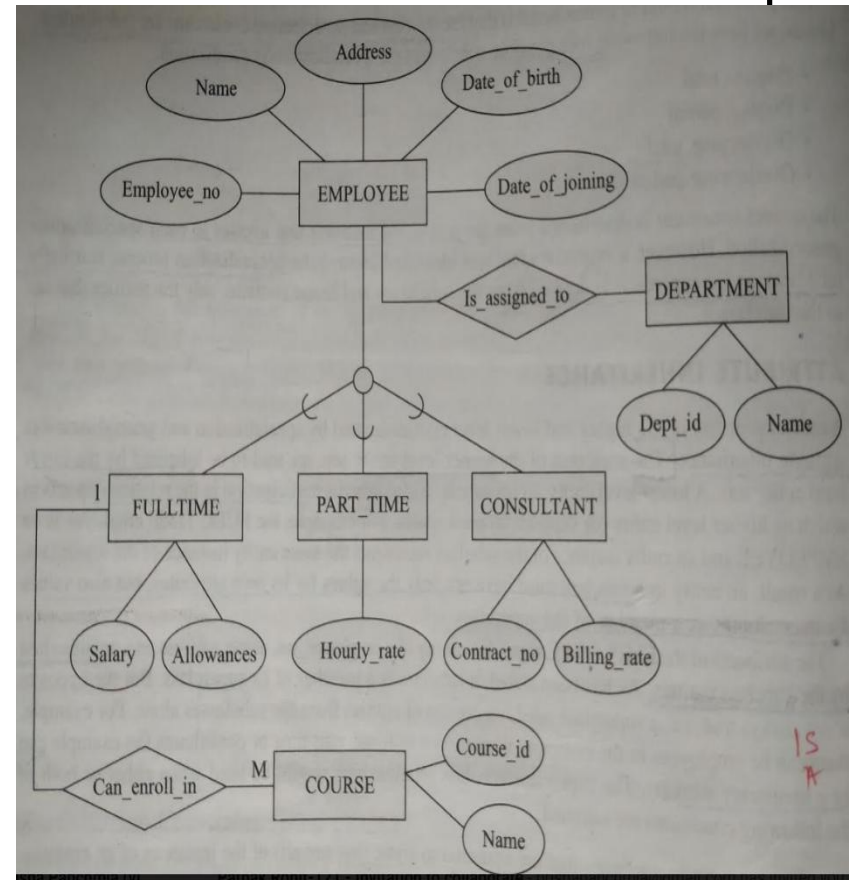
- **Constraints on Generalization**
- **Completeness constraint** on a generalization or specialization, specifies whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within the generalization/specialization. This constraint may be one of the following:
 - **Total generalization or specialization.** Each higher-level entity must belong to a lower-level entity set. The *account generalization is total: All account entities must be either a savings account or a checking account.*
 - **Partial generalization or specialization.** Some higher-level entities may not belong to any lower-level entity set. The work team entity sets illustrate a partial specialization. Since employees are assigned to a team only after 3 months on the job, some *employee entities may not be members of any of the lower-level team entity sets.*

Four possible constraints on Generalization/Specialization

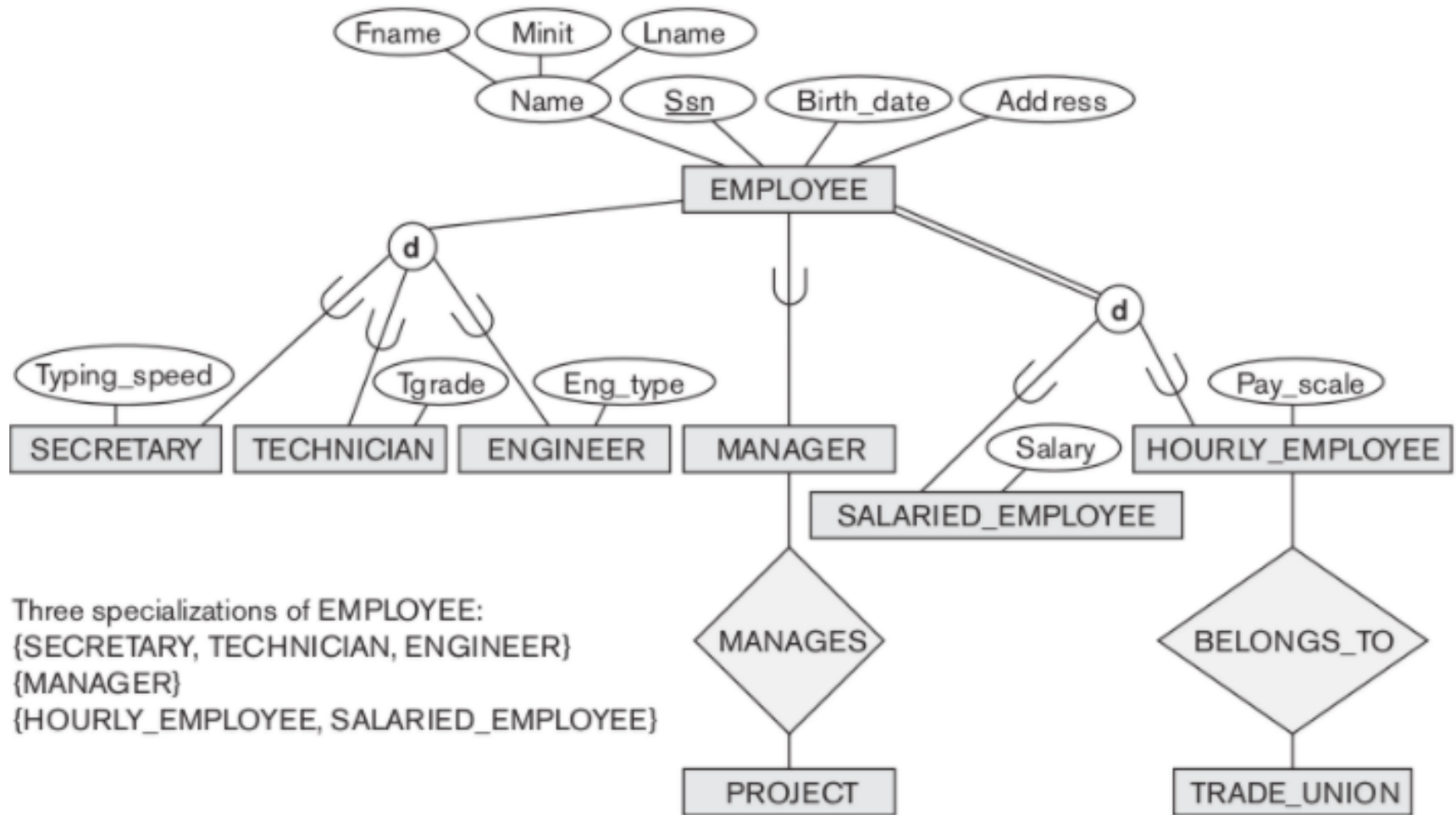
- Disjoint, Total
- Disjoint, Partial
- Overlapping ,Total
- Overlapping ,Partial

Employee Superclass/Subclass relationship

- The attributes of the higher level entity sets are said to be inherited by lower level entity set
- The Subclass also inherits participation in the relationship sets in which superclass participates- (Subclasses inherits relationship of superclass)



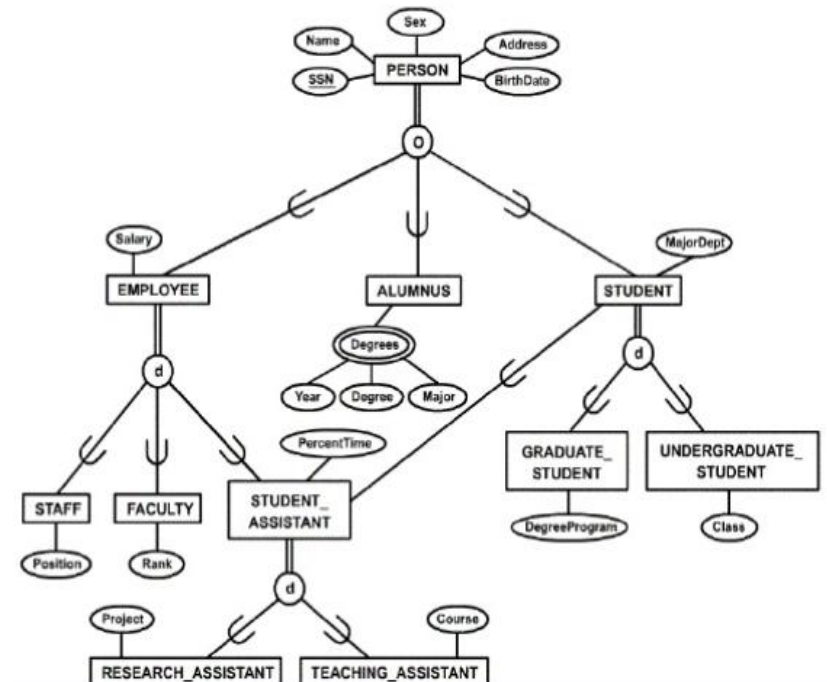
EER diagram notation to represent subclasses and specialization



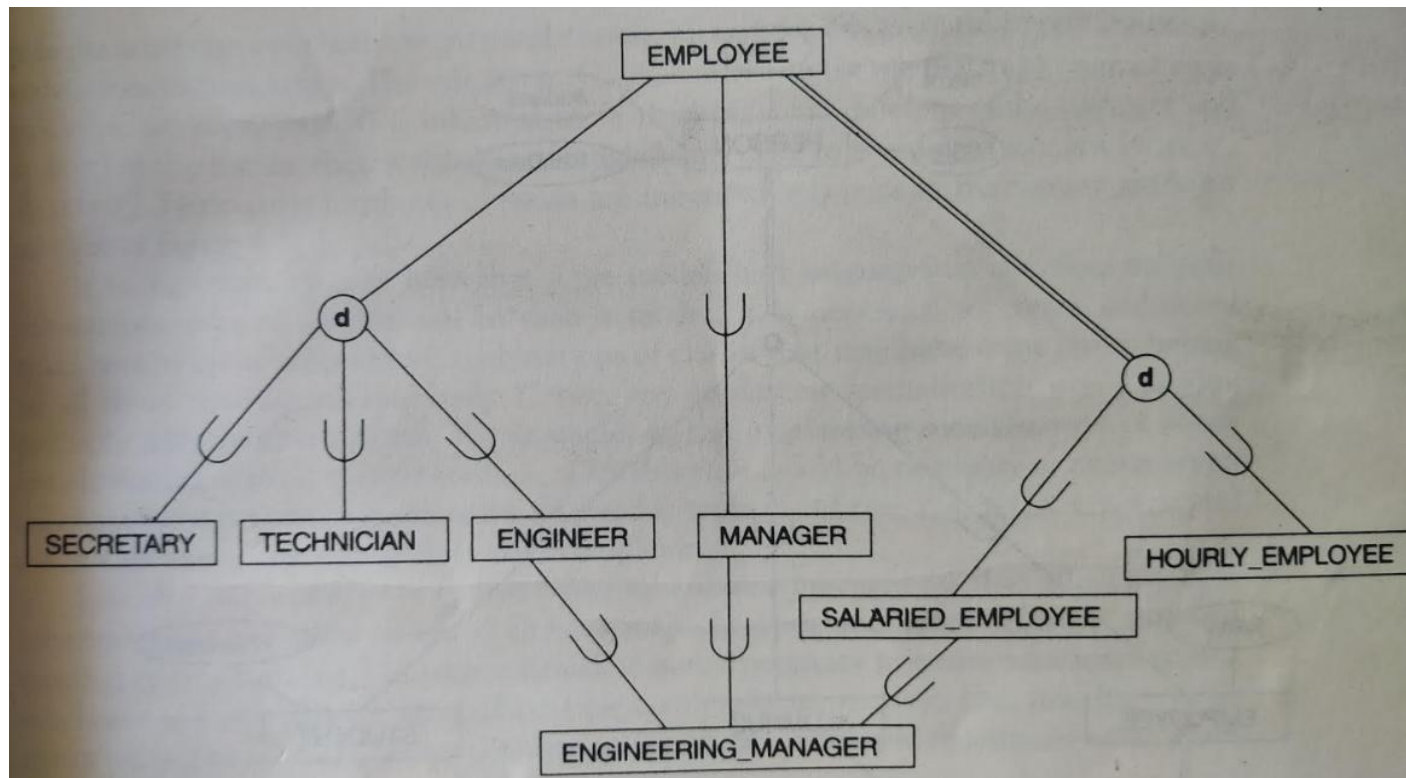
Specialization/Generalization hierarchy and Lattices

- A specialization **hierarchy** has the constraint that every subclass has only one parent, which result in **tree** structure
- A specialization **lattice** can be a subclass in **more than one class/subclass** has more than one level relationship
- Eg: Student_Assistant
- A Subclass with more than one Superclass is called **Shared subclass**
- This leads to the concept of **Multiple Inheritance**- inherits all the attributes and relationships of multiple classes

Specialization / Generalization Lattice Example (UNIVERSITY)

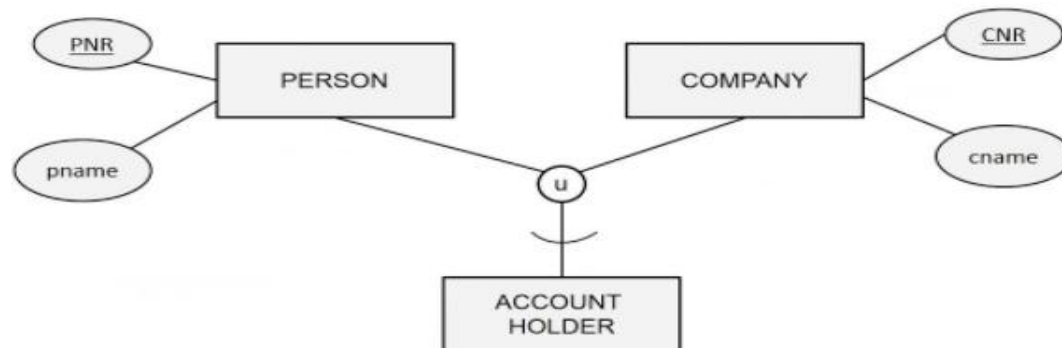


Contd...



Categorization or Union

- The modeling of a single subclass with a relationship that involves more than one distinct superclass is called **Categorization**
- A subclass having **more than one superclass-** is called a **category** and a process of defining a category is called Categorization
- A subclass will represent a collection of objects that is a subset of the **UNION** of distinct entity types

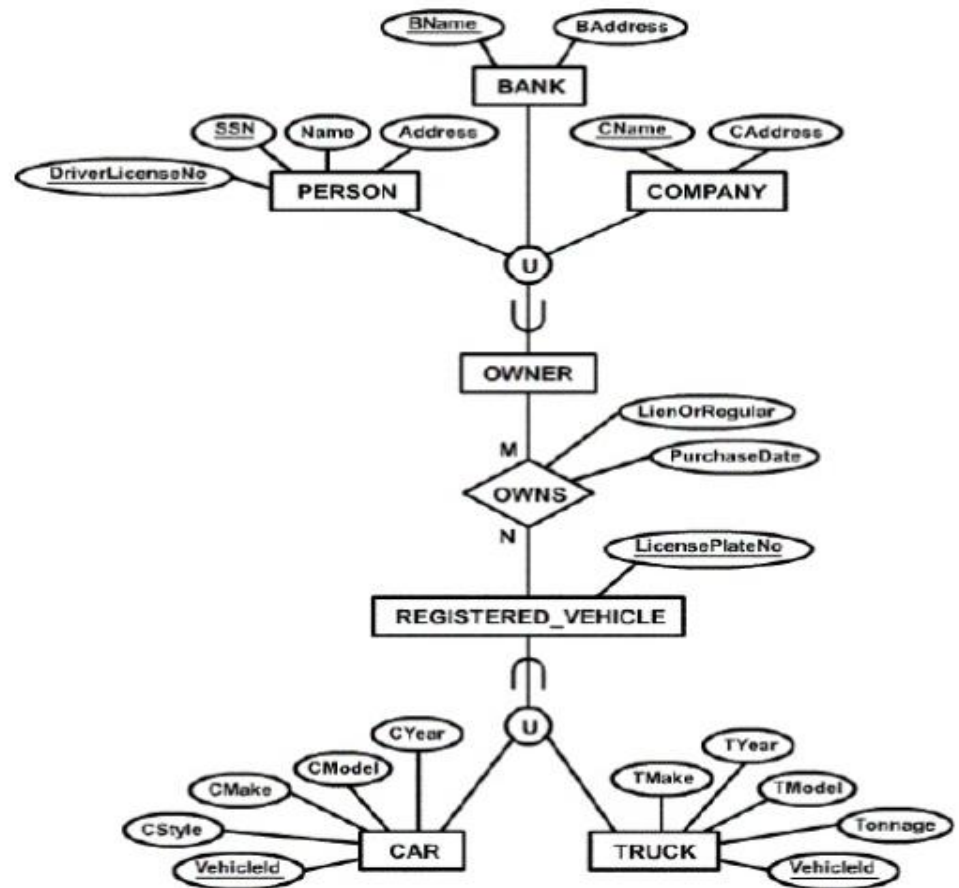


Contd...

- The line connecting the category subclass to the categorization circle has the subset symbol($\subset$) and the circle itself contains the union symbol($\cup$)
- The category subclass has **selective inheritance**. The category will inherit only the attributes of **one of the superclass at a time**.
- A category can be subdivided depending on total or partial participation

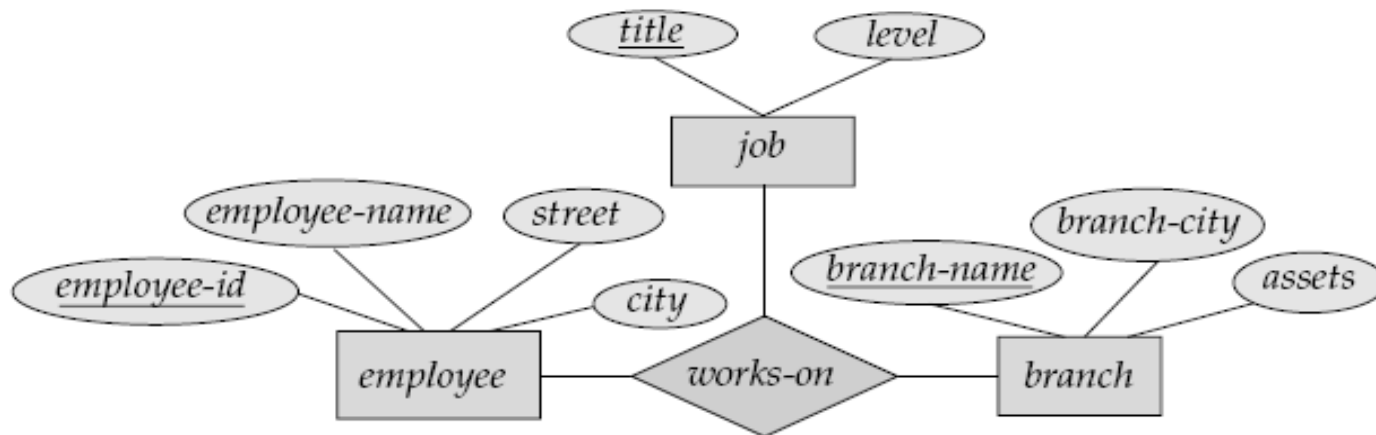
Example of categorization Union type

- A Superclass- Company, Person, bank are connected to circle with **U** which stands for **set union** operation
- Eg: A Category owner is a subclass of union of superclasses -three entity sets: Company, bank, person
- Owner must exist in only one of the superclass



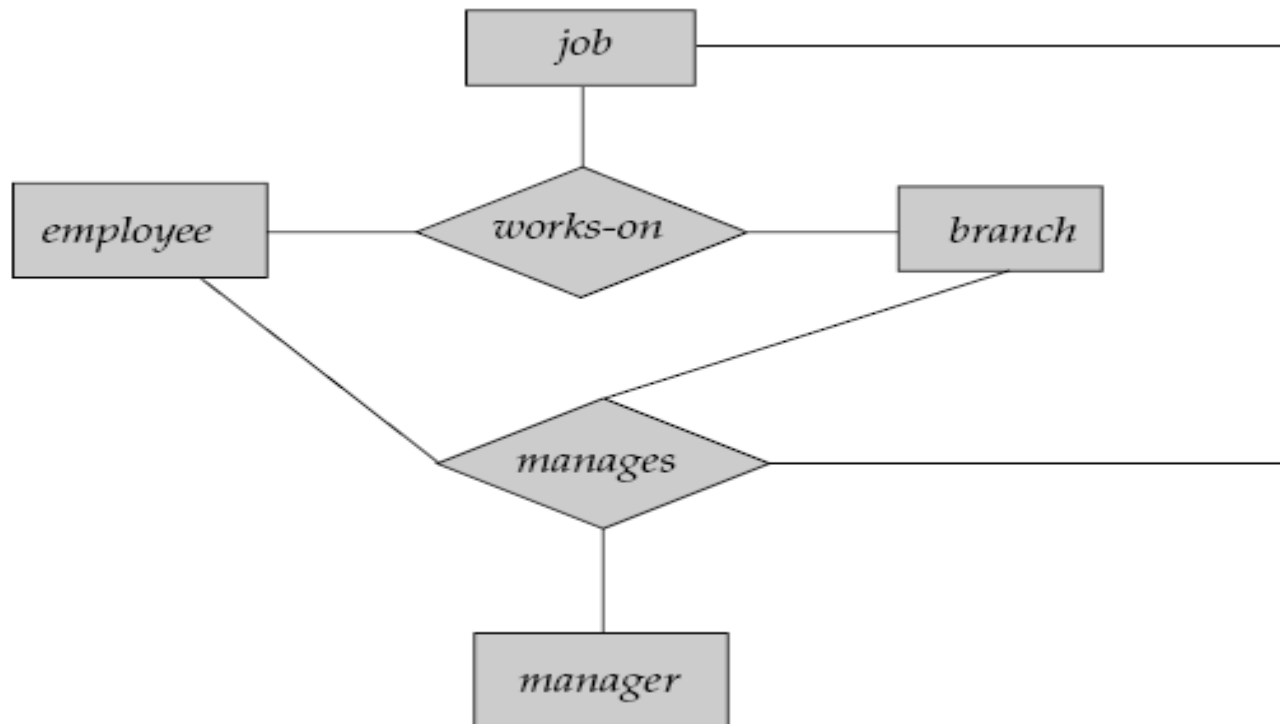
Aggregation

- One limitation of the E-R model is that it cannot express relationships among relationships
- Aggregation is an abstraction through which relationships are treated as **higher-level entities**
- Consider the ternary relationship *works-on*, between a *employee*, *branch*, and *job*



Contd...

- Suppose we want to record managers for (*employee*, *branch*, *job*) combinations. Let us assume that there is an entity set *manager*.
- One alternative for representing this relationship is to create a quaternary relationship *manages* between *employee*, *branch*, *job*, and *manager*.
- Using the basic E-R modeling constructs, we obtain the E-R diagram

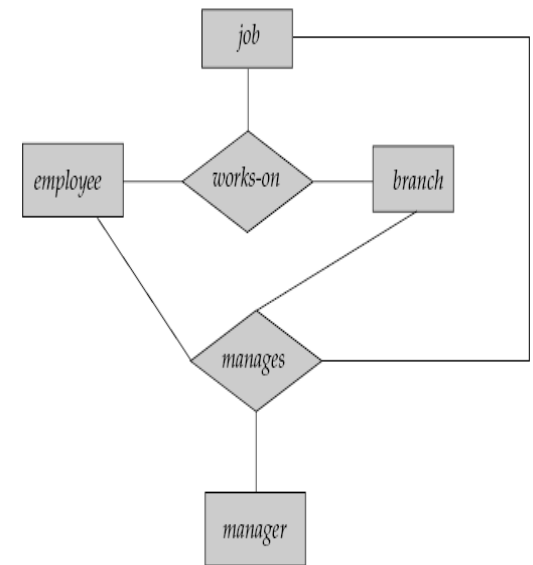


Contd...

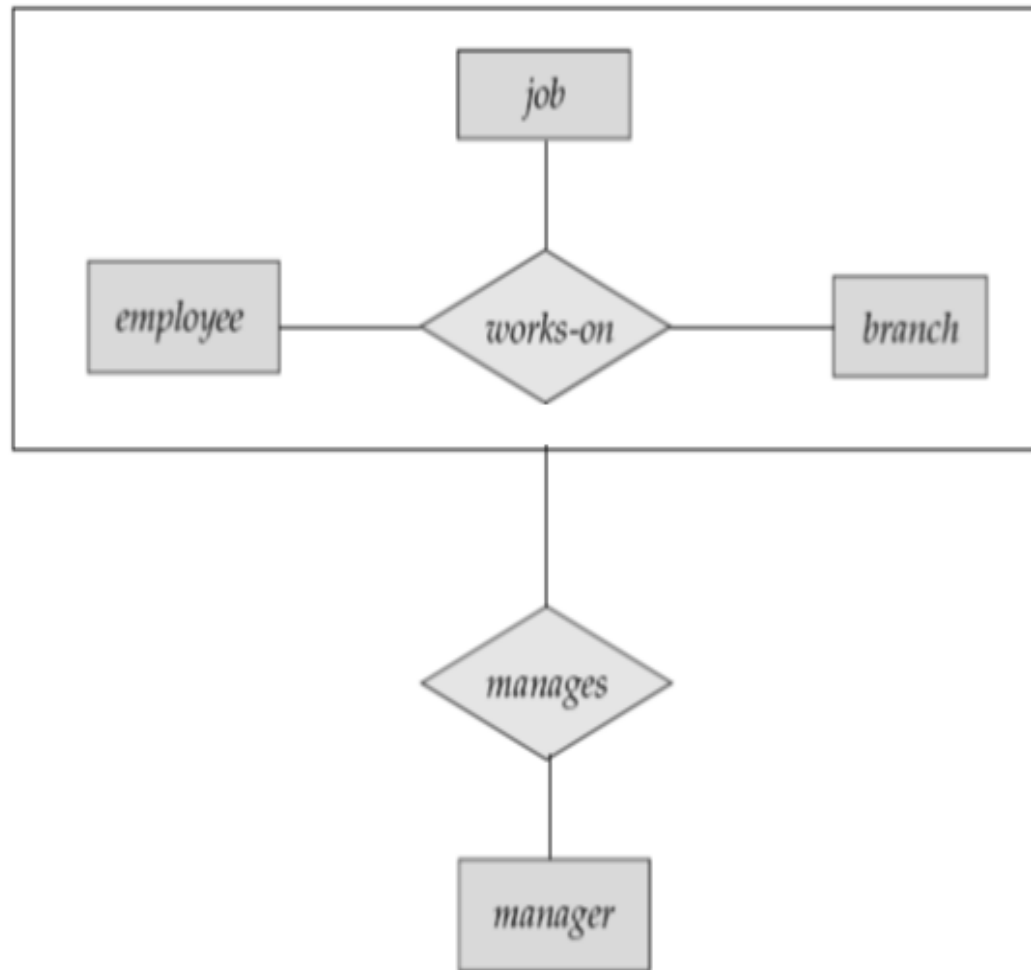
- Relationship sets *works_on* and *manages* represent overlapping information
- Every *manages* relationship corresponds to a *works_on* relationship
- However, some *works_on* relationships may not correspond to any *manages* relationships

– So we can't discard the *works_on* relationship

- Eliminate this redundancy via *aggregation*
- Treat relationship as an abstract entity
- Allows relationships between relationships
- Abstraction of relationship into new entity



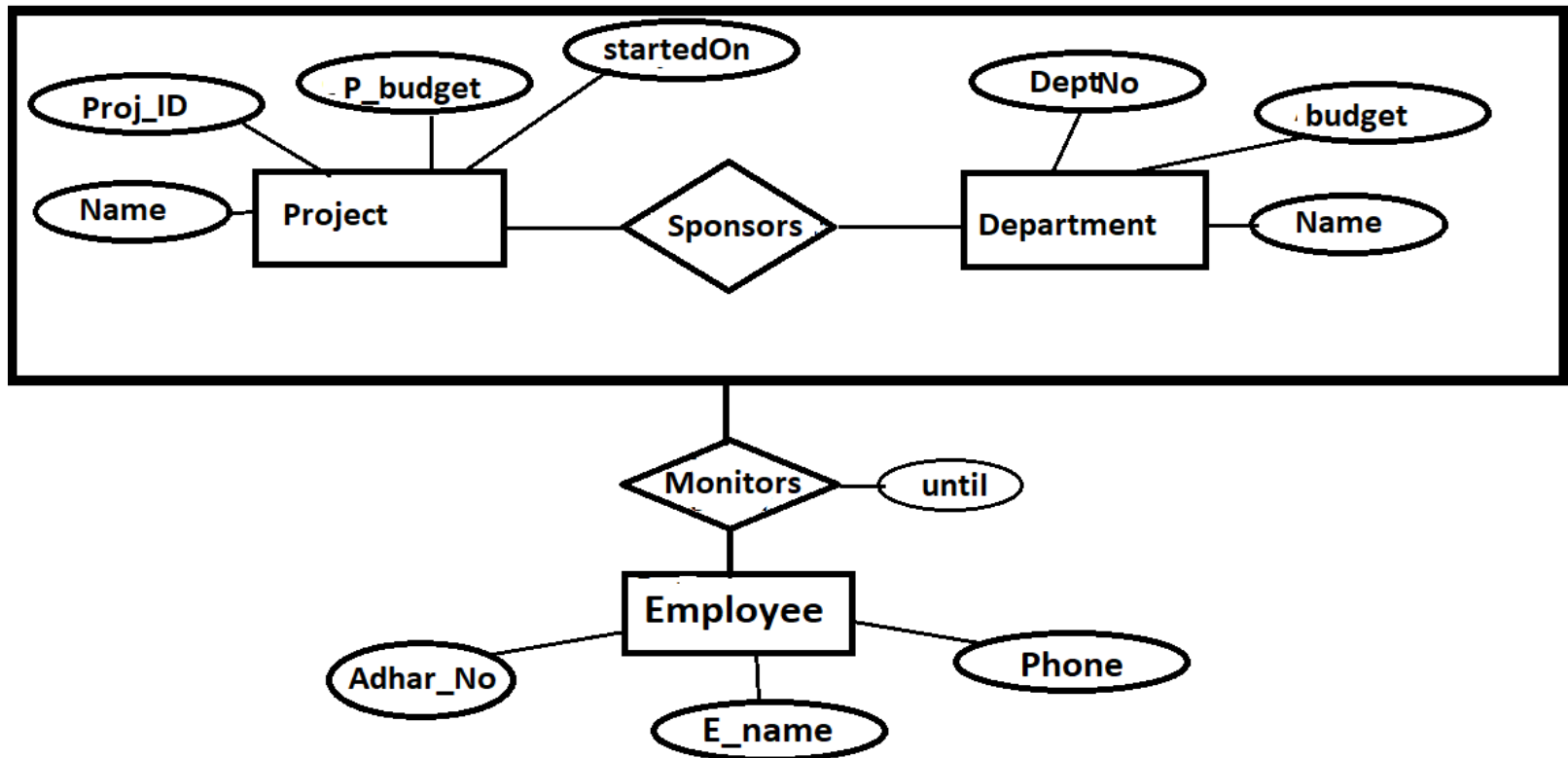
- Without introducing redundancy, the following diagram represents:
 - An employee works on a particular job at a particular branch
 - An employee, branch, job combination may have an associated manager



itd...

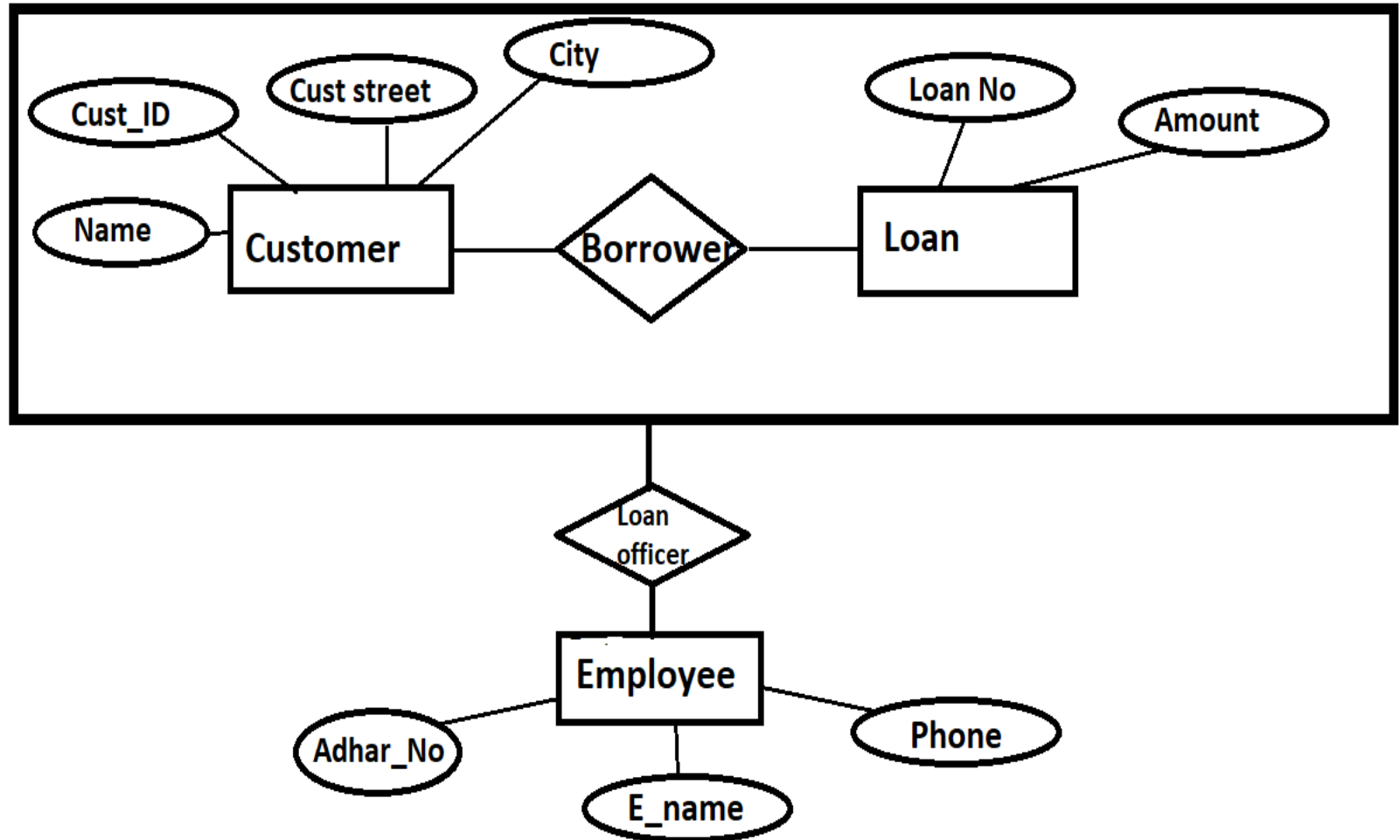
Example 2:

- Aggregation is an abstraction, relationships are treated as higher-level entities



Contd...

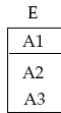
- Example 3:**



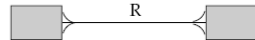
Extended E-R Features

- Alternative E-R Notations**

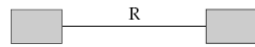
entity set E with
attributes A1, A2, A3
and primary key A1



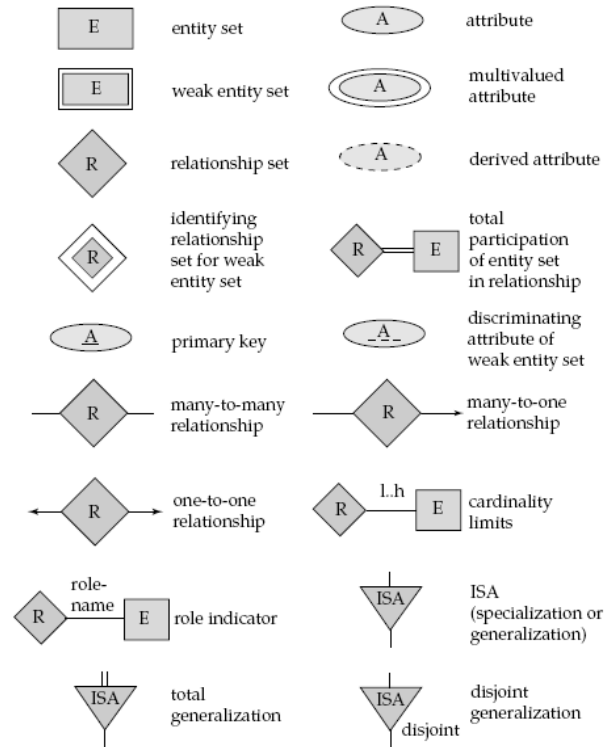
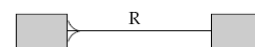
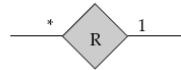
many-to-many
relationship



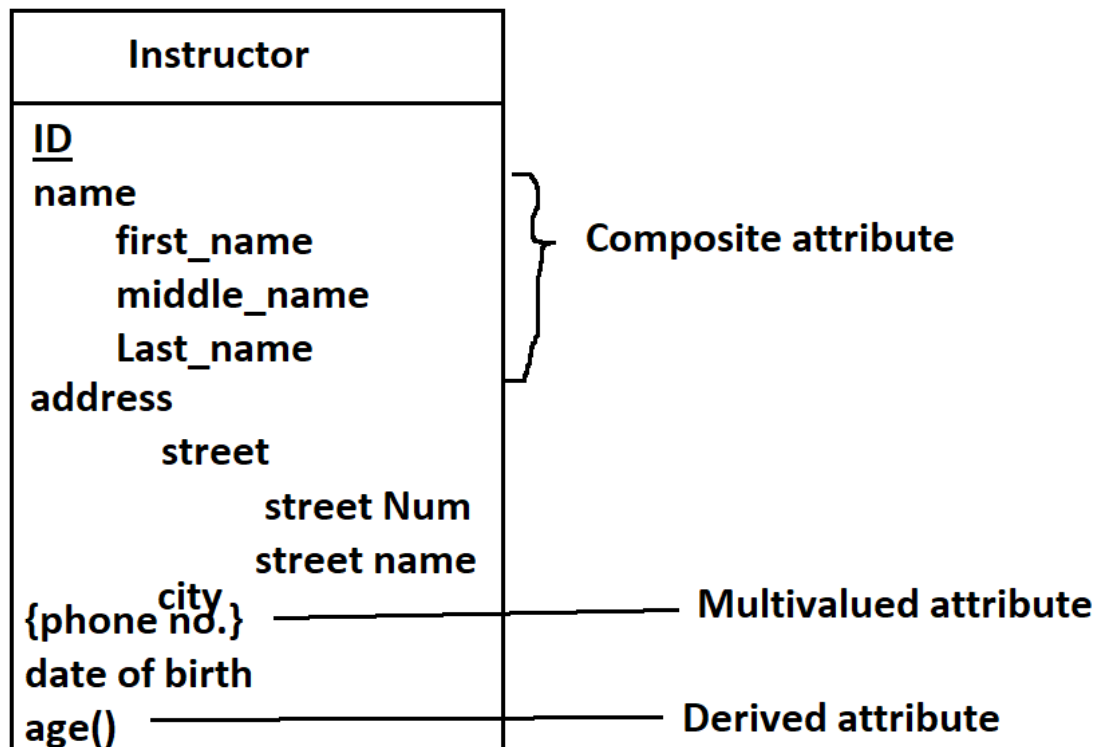
one-to-one
relationship



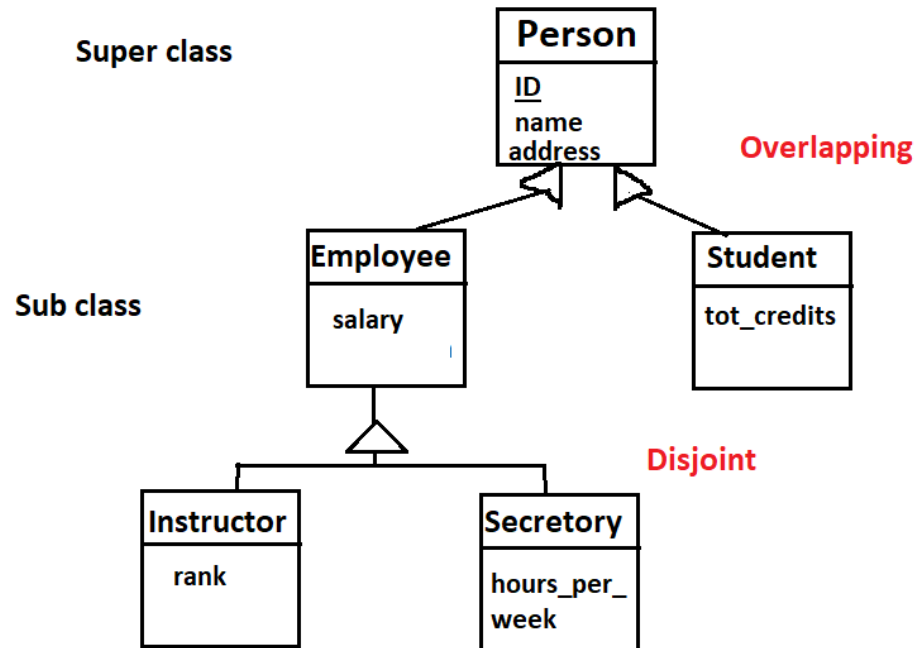
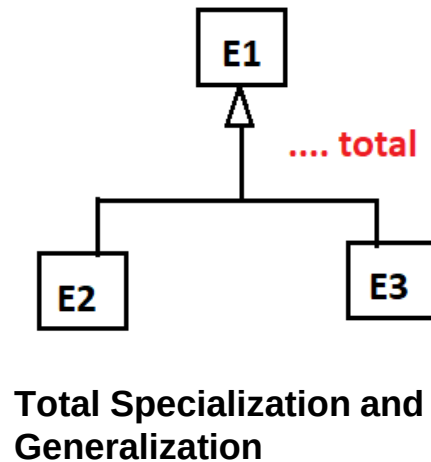
many-to-one
relationship



One more way for Entity set representation in EER diagram



One more way for Disjoint and overlapping in Specialization and Generalization



Database Design for Banking Enterprise

- The bank is organized into branches. Each branch is located in particular city and is identified by a unique name. The bank monitors the assets of each branch.
- Bank customers are identified by their customer_id values. The bank store each customer's name and street and city where the customer lives. Customers may have account and can take out the loans. A customer may be associated with a particular banker who may act as a loan officer or personal banker for that customer.
- Bank employees are identified by their employee_id values. The bank administration stores the name and the telephone number of each employee, the name of employee's dependents and employee_id number of employee's manager. The bank also keeps track of the employees' start date and thus the length of employment.
- Bank offers two types of account- saving and checking account. Accounts can be held by more than one customer and a customer can have more than one account. Each account is assigned unique account number. The bank maintains a record of each account' balance and most recent date on which the account was accessed each customer holding the account. Each saving account has an interest rate and overdrafts are recorded for each checking account.
- The bank provides its customer with loans. A loan is originates at particular branch and can be held by one or more customers. A loan is identified by a unique loan number. For each loan, bank keeps track of the loan amount and the loan payment. Although a loan payment number does not uniquely identify a particular payment among those for all bank's loan, a payment number does identify a particular payment for a specific loan. The date and amount are recorded for each payment.

Contd...

- **Entity Sets for the Bank Database**
- The *branch* entity set, with attributes *branch-name*, *branch-city*, and *assets*.
- The *customer* entity set, with attributes *customer-id*, *customer-name*, *customer-street*; and *customer-city*. A possible additional attribute is *banker-name*.
- The *employee* entity set, with attributes *employee-id*, *employee-name*, *telephone-number*, *salary*, and *manager*. Additional descriptive features are the multivalued attribute *dependent-name*, the base attribute *start-date*, and the derived attribute *employment-length*.
- Two account entity sets—*savings-account* and *checking-account*—with the common attributes of *account-number* and *balance*; in addition, *savings-account* has the attribute *interest-rate* and *checking-account* has the attribute *overdraft-amount*.
- The *loan* entity set, with the attributes *loan-number*, *amount*, and *originating-branch*.
- The weak entity set *loan-payment*, with attributes *payment-number*, *payment-date*, and *payment-amount*.

Contd...

- **Relationship Sets Designation**
- *borrower*, a many-to-many relationship set between *customer* and *loan*.
- *loan-branch*, a many-to-one relationship set that indicates in which branch loan originated.

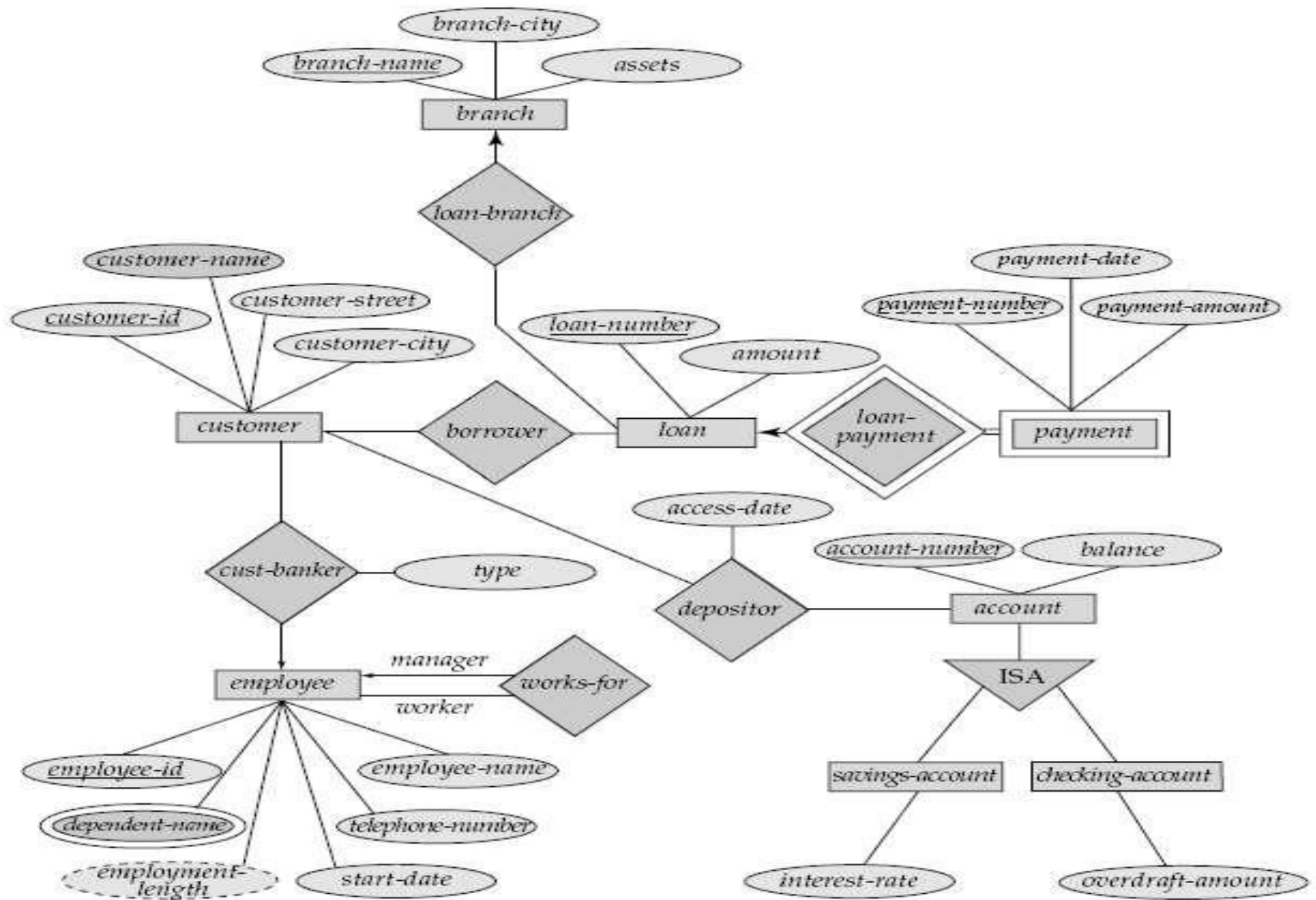
Note that this relationship set replaces the attribute *originating-branch* of the entity set *loan*.

- *loan-payment*, a one-to-many relationship from *loan* to *payment*, which documents that a payment is made on a loan.
- *depositor*, with relationship attribute *access-date*, a many-to-many relationship set between *customer* and *account*, indicating that a customer owns an account.

Contd...

- *cust-banker*, with relationship attribute *type*, a many-to-one relationship set expressing that a customer can be advised by a bank employee, and that a bank employee can advise one or more customers. Note that this relationship set has replaced the attribute *banker-name* of the entity set *customer*.
- *works-for*, a relationship set between *employee* entities with role indicators *manager* and *worker*; the mapping cardinalities express that an employee works for only one manager and that a manager supervises one or more employees. Note that this relationship set has replaced the *manager* attribute of *employee*.

E-R Diagram: Database Design for Banking Enterprise



Example1 :

- Design a database to keep track of information for an art museum. Assume that the following requirements were collected
- The museum has a collection if ART_OBJECT. Each ART_OBJECT has a unique ID_NO, an Artist(if known), a year(when it was created, if known), a title and description. The art Objects are categoized based on their type. There are 3 main types : PAINTING, SCULPTURE AND STATUE, plus another type called OTHER to accommodate objects that do not fall into one of the 3 main types.

Example2:

- Construct an EER diagram for the following description:
- “ A university maintains records of its students and the programmes in which they have enrolled. It stores student_id,name,address and phone number of a student and programmes code, program name and duration of a programme. A student is either a full time or a part time student(only one of the types). A student can register for many programmes and a programmes can have many students.

Summary of Symbols Used in E-R Notation

